

RFC (v2): A native qang module for Qiskit and Cirq

Vicente Humberto Monteverde

Universidad del Museo Social Argentino (UMSA), Argentina

ORCID 0000-0001-8884-4811

Revised 30 September 2026. Version 1: 19 September 2026, doi:10.5281/zenodo.22845682

Abstract

This request for comments proposes an optional, additive module for Qiskit and Cirq that lets circuit authors parameterize single-qubit rotations by the population bias they produce, $qg_Z = \langle \sigma_z \rangle$ (the qang unit [1]), instead of by a raw angle, together with a small set of estimation and analysis helpers. Version 2 revises version 1 against ten days of further work in the reference implementation. It adds a signed gate that removes the arccos range limit; the symmetry filter as an explicit operation on outcome distributions; and a noise-free compile-time screen, the sector exposure, that ranks compilations by how much relaxation error the filter will let through. It narrows the scope where the evidence says so: the qudit generalization is withdrawn from the proposal, since pre-registered tests found no qutrit advantage for the qg quantities themselves, and error-correction decoder rules are kept out of the SDKs. Every proposed function exists and is tested in the reference package; nothing in the proposal has yet been validated on hardware.

1 What changed since version 1

- **Reference implementation.** The package is `qang` (version 0.3.0) at <https://github.com/Viny2030/qang>, MIT license, with 1103 tests run in continuous integration on Python 3.9–3.12. Version 1 pointed to an earlier notebook repository; the names below refer to the package.
- **Added:** `SignedRQangGate` (Section 3.2); `qang.sectors` with `filter_distribution` and `sector_exposure` (Section 3.3); signed and natural-gradient steps in qg coordinates; interval estimators for qg_Z (Bayesian, Wilson, delta method).
- **Withdrawn:** the $su(d)$ qudit module, the Householder and Grover-diffuser constructors, and anything tied to shadows or Grover read-outs (Section 4).
- **Kept out of the SDKs:** the qg-aware surface-code decoder rules. They belong to decoder libraries, not to circuit SDKs.

2 The unit in one paragraph

For $|\psi(\theta)\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$, the polar qang is $qg_Z = \cos \theta = \langle \sigma_z \rangle \in [-1, 1]$, and the entropic qang is $qg_S = H((1 + qg_Z)/2)$, the binary Shannon entropy of the outcome. Neither is a new physical quantity; the unit gives them a name, a value type with round-trip conversions, and a gate that takes them as its parameter [1]. The case for upstreaming is ergonomic: one round-trip-safe value instead of scattered arccos and entropy conversions.

3 Proposed API

3.1 Gates

The reference implementation provides three Qiskit gates, each decomposing to an existing basis gate, so nothing else in the transpiler or on hardware needs to change:

```
from qang import Qang
from qang.qiskit_gate import RQangGate, FullRQangGate, SignedRQangGate

qc.append(RQangGate(0.7071), [0]) # RY(arccos qg_Z)
qc.append(FullRQangGate(Qang(0.7071, phi=0.3)), [0]) # U(theta, phi, 0)
qc.append(SignedRQangGate(0.7071, sign=-1), [0]) # RY(-arccos qg_Z)
```

The Cirq counterparts are `rqang_gate` (built on `cirq.ry`) and `full_rqang_gate` (a `cirq.MatrixGate` matching `FullRQangGate` bit for bit). A signed Cirq gate is a one-line addition and is proposed with the rest.

3.2 The signed gate

qg_Z alone fixes θ only on $[0, \pi]$: the states with θ and $-\theta$ have the same qg_Z . The pair (qg_Z, s) with $s = \text{sign } qg_X = \pm 1$ covers the full great circle, and `SignedRQangGate` applies $R_Y(s \arccos qg_Z)$. For variational circuits the reference package also provides steps that keep track of s and reflect through a pole instead of stopping there (`signed_qg_step`, `signed_natural_qg_step`). In the tests behind them, every unsigned qg update on H_2 stopped at the Hartree–Fock point, because the optimum sits at $\theta \approx -0.22$; both signed steps reached the exact energy. The honest corollary is that optimizing in qg coordinates gains nothing over optimizing the angle: the natural metric in qg reduces to θ . The signed gate is proposed for state preparation and parameter binding, not as a faster optimizer.

3.3 The filter as an operation, and a compile-time screen

Many workloads conserve the number of excitations (Hamming weight): particle-number-conserving chemistry ansätze, XY mixers, Heisenberg-type dynamics. Discarding shots outside the conserved sector is a known, cheap mitigation (symmetry verification [2, 3]); in qg terms it is the shot-level form of the register-mean witness of the companion preprint [4]. The reference package exposes it as an explicit operation:

```
from qang.sectors import filter_distribution, sector_exposure

p_kept, kept_fraction = filter_distribution(counts, n_qubits=6, weight=3)
exp = sector_exposure(circuit, weight=3)
# -> {"mean": ..., "max": ..., "per_gate": [...]}
```

`sector_exposure` answers a question that matters before any noisy run: how much of the relaxation (T1) error will the filter catch for this particular compilation? A decay that happens while part of the state is outside the sector can be rotated back into it by later gates and pass the filter. The exposure is the population outside the sector right after each two-qubit gate, computed on a noiseless statevector. In the tests behind it (Heisenberg chain, 16 compilation and time-step points), its rank correlation with the share of T1 error the filter lets through was 0.80, a pre-registered threshold met without margin. Every compilation with zero exposure (number-conserving two-qubit gates) removed at least 98% of the T1 error, while 3-CNOT compilations, with exposure 0.42, let about half through. We propose it as an optional analysis pass (for Qiskit, a transpiler `AnalysisPass` that writes the exposure to the property set), usable to choose between compilations. It needs a statevector, so it is limited to circuits of about 20 qubits; it ranks compilations and does not predict error magnitudes.

3.4 Estimation

`qang.statistics` provides qg_Z estimates with intervals from k_0 zeros in N shots: a Bayesian posterior under the Haar prior (uniform in qg_Z), the Wilson score interval, and the delta method, plus the covariance of several qg_Z values from one set of counts. The Bayesian and Wilson intervals remain informative at the poles, where the delta method collapses. We propose them under a single `qg_estimate(k0, n_shots, method=...)` entry point.

4 Scope, revised by evidence

Several items of version 1 were tested since, and the proposal follows the results, including the negative ones:

- **Qudits.** Version 1 proposed an $\text{su}(d)$ generalization of qg_Z as a reference module. Pre-registered surface-code tests with leakage to $|2\rangle$ found that a three-level readout helps only in specific leakage channels, and then as an extra input to a decoder, not as a new qg quantity [5]. For a simulated transmon CZ, the leakage channel left the three-level readout without useful information. The qudit module is withdrawn.
- **Shadows and Grover.** Direct Z-basis measurement was $3\text{--}41\times$ cheaper in shots than classical shadows for the qg quantities, and per-qubit qg readings lost to the plain outcome histogram in Grover search. The Householder and Grover-diffuser constructors of version 1 are therefore dropped: they add a name to a well-known matrix without a demonstrated use.
- **Decoders.** A qg-aware reweighting of a surface-code decoder, switched by the register-mean witness, gained $1.6\text{--}3.3\times$ in logical error under simulated relaxation-dominated noise, and two rules followed: a Bayesian weight for leakage flags, and a $1 - h$ scaling of priors on erasure qubits with heralding efficiency h [4, 5]. These concern decoders (for example the weights passed to PyMatching [6]), not circuit construction, and are not proposed for Qiskit or Cirq.

5 Adoption path

1. **Standalone package** (current). `qang` with Qiskit, Cirq and PennyLane as optional extras; publication on PyPI is the next step.
2. **Ecosystem listing.** The Qiskit ecosystem page and a Cirq contrib-style listing, with no review burden on either core team.
3. **Upstream proposal.** A Qiskit Enhancement Proposal and a Cirq design-doc issue, each scoped to the gates of Section 3.1 and, as a separate item, the `sector_exposure` analysis pass.
4. **Reference code as the pull request.** Every item above is already implemented and tested in the package.

6 Open questions and risks

- **Naming.** “qang” is not a standard term; maintainers may prefer `RYGate.from_z_bias` and helper functions. The ergonomic case (Section 2) has to carry the proposal, not the name.

- **Hardware.** All validation so far is simulation, including IonQ’s cloud simulator and its forte-1 noise model. A QPU run is prepared but not yet executed; reviewers will likely ask for it.
- **Exposure is a screen, not a model.** Its evidence is one rank correlation at 0.80 on small circuits, one noise type, and a statevector cost that grows exponentially.
- **Duplicate surface.** Both SDKs already have rich angle machinery. The proposal is strictly additive: every gate decomposes to an existing one, and nothing becomes a basis gate.
- **Maintainer bandwidth.** Step 1 delivers value regardless of upstream acceptance.

References

- [1] Vicente Humberto Monteverde. The Qang (qg): A unified angular-probability unit and metric for parametric quantum circuit design. 2026. doi: 10.5281/zenodo.22832150.
- [2] Xavier Bonet-Monroig, Ramiro Sagastizabal, M. Singh, and Thomas E. O’Brien. Low-cost error mitigation by symmetry verification. *Physical Review A*, 98:062339, 2018.
- [3] Sam McArdle, Xiao Yuan, and Simon Benjamin. Error-mitigated digital quantum simulation. *Physical Review Letters*, 122:180501, 2019.
- [4] Vicente Humberto Monteverde. The register-mean polar bias as a symmetry witness: error mitigation, constrained optimization and qubit characterization in qg units. *Zenodo preprint*, 2026. doi: 10.5281/zenodo.23036963.
- [5] Vicente Humberto Monteverde. When does a three-level readout help? leakage flags, erasure qubits and the qg T1-aware decoder, 2026. Companion note, `manuscript/leakage/` in the repository.
- [6] Oscar Higgott. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3):16, 2022. doi: 10.1145/3505637.